



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de

HONORIS UNITED UNIVERSITIES

Cahier de charge

NASA TELEMTRY SYSTEM

Java avancée

Réalisé par :

Nom:

Abdessamad El Jazouly

William Fomin

Aliaa Chibel

Ayman Lafnoune

Année universitaire : 2024 / 2025

Cahier des Charges – NASA Telemetry System

1. Présentation générale

Le projet **NASA Telemetry System** est une application Java simulant la communication entre des **satellites** et une **station terrestre**.

Le système permet de :

- Générer des données de télémétrie côté satellite.
- Transmettre ces données au serveur via **Sockets TCP**.
- Recevoir, parser et stocker les données dans une **base SQL** et des fichiers CSV.
- Optionnellement visualiser les données via une interface JavaFX.

L'application est conçue pour être **modulaire, maintenable et évolutive**, avec une séparation claire entre client, serveur, modèles et utilitaires.

2. Objectifs du projet

2.1 Objectif principal

Simuler un environnement de télémétrie spatiale complet, capable de gérer plusieurs satellites et de stocker leurs données de manière fiable et organisée.

2.2 Objectifs spécifiques

- Gérer plusieurs satellites simultanément.
- Générer automatiquement des données de télémétrie : altitude, vitesse, batterie, température, orbite.
- Transmettre les données via Sockets TCP.
- Parser les messages et transformer en objets métier (Telemetry).
- Stocker les données dans une base SQL et dans des fichiers CSV.
- Fournir une architecture claire pour faciliter les extensions futures.
- Optionnel : interface graphique JavaFX pour visualiser les données.

3. Fonctionnalités

3.1 Satellite (client)

- Génère les télémétries via SatelliteGenerator.

- Se connecte au serveur via SatelliteClient.
- Envoie les données à intervalle régulier.
- Les données incluent : altitude, vitesse, batterie, température, orbite.
- Peut simuler plusieurs satellites en parallèle.

Exemple de message :

SAT-1; altitude=400; vitesse=7500; batterie=85; temperature=22; orbite=LEO

3.2 Serveur (ServerMain)

- Écoute sur un port configurable.
- Accepte les connexions de plusieurs satellites.
- Crée un thread SatelliteHandler par connexion.
- Reçoit les messages et les passe à MessageParser.
- Transmet les objets Telemetrie au module de stockage.

3.3 Modèle et DAO

Modèles

- Satellite.java : informations sur le satellite (nom, statut, date de lancement).
- Telemetrie.java : télémétrie envoyée par un satellite (altitude, vitesse, batterie, température, orbite, timestamp).

DAO

- SatelliteDAO.java : CRUD pour les satellites.
- TelemetrieDAO.java : insertion des télémétries dans la base SQL.

3.4 Stockage des données

Base SQL

- Table satellite : informations des satellites.

Champ	Type	Description
id	INT	Clé primaire, auto-incrémentée
name	VARCHAR(50)	Nom du satellite (ex : SAT-1, SAT-2)
launch_date	DATE	Date de lancement du satellite (optionnel)
status	VARCHAR(20)	Statut du satellite (ex : connecté / déconnecté)
orbit	VARCHAR(20)	Type d'orbite (LEO, MEO, GEO, etc.)
description	TEXT	Informations complémentaires (optionnel)

- Table telemetry : télémétries reçues.

Champ	Type	Description
id	int	Clé primaire
satellite_id	int	Référence vers Satellite
altitude	double	Altitude en km
vitesse	double	Vitesse en km/h
batterie	double	Niveau de batterie (%)
température	double	Température (°C)
orbite	varchar	Type ou position de l'orbite
timestamp	datetime	Date/heure de réception

CSV

- Géré par CSVLogger.java.
- Enregistrement continu de chaque télémétrie.

Format :

timestamp,satellite,altitude,vitesse,batterie,temperature,orbite

3.5 Interface utilisateur

- DashboardMain.java et DashboardController.java.
- Affiche les satellites connectés et les télémétries.
- Graphiques temps réel pour altitude, vitesse, batterie et température.
- Lecture et analyse des fichiers CSV.

4. Architecture du projet

Le projet NASA Telemetry System est organisé de manière modulaire afin de séparer les responsabilités :

- model/ : contient les classes métier Satellite et Telemetrie.
- database/ : gère la connexion à la base SQL via Database.java.
- dao/ : classes DAO pour accéder aux données (SatelliteDAO, TelemetrieDAO).
- server/ : serveur principal, gestion des satellites connectés (ServerMain, SatelliteHandler, MessageParser).
- client/ : simulateur de satellites (SatelliteClient, SatelliteGenerator).
- ui/ : interface JavaFX optionnelle pour visualiser les données (DashboardMain, DashboardController).
- utils/ : utilitaires pour CSV et configuration (CSVLogger, Config).
- lib/ : bibliothèques externes nécessaires.

Cette organisation garantit une clarté, une maintenabilité et une évolutivité du projet.

Chaque module est indépendant, ce qui facilite les tests et la mise à jour du code.

Le multithreading dans le serveur permet de gérer plusieurs satellites simultanément.

La séparation DAO / modèles assure une persistance propre et cohérente des données.

Le client et le serveur communiquent via des sockets TCP pour simuler la télémétrie en temps réel.

Les utilitaires permettent l'enregistrement automatique des données dans des fichiers CSV.

L'architecture est conçue pour être facilement extensible à de nouveaux satellites ou types de mesures.

NASA TELEMETRY SYSTEM/

```
|— src/
|   |— model/
|   |   |— Satellite.java
|   |   └─ Telemetrie.java
|   |— database/
|   |   └─ Database.java
|   |— dao/
|   |   |— SatelliteDAO.java
|   |   └─ TelemetrieDAO.java
|   |— server/
|   |   |— ServerMain.java
|   |   |— SatelliteHandler.java
|   |   └─ MessageParser.java
|   |— client/
|   |   |— SatelliteClient.java
|   |   └─ SatelliteGenerator.java
|   |— ui/
|   |   |— DashboardMain.java
|   |   └─ DashboardController.java
|   └─ utils/
|       |— CSVLogger.java
|       └─ Config.java
|— lib/
```

5. Contraintes techniques

- Java 8+ / Java 17
- Sockets TCP pour communication client-serveur
- Programmation orientée objet
- DAO pour persistance en base SQL
- Multithreading pour gérer plusieurs satellites
- CSV pour stockage secondaire
- Architecture modulaire respectant la structure définie
- Optionnelle : JavaFX pour interface graphique

6. Scénario de fonctionnement

1. Démarrage du serveur (ServerMain) → écoute des connexions.
2. Connexion d'un ou plusieurs satellites (SatelliteClient).
3. Chaque satellite envoie ses données périodiquement.
4. SatelliteHandler reçoit le message → MessageParser convertit en Telemetrie.
5. Les données sont stockées :
 - Dans la **base SQL** via DAO
 - Dans le **CSV** via CSVLogger
6. Optionnel : l'UI affiche les données en temps réel.

7. Livrables

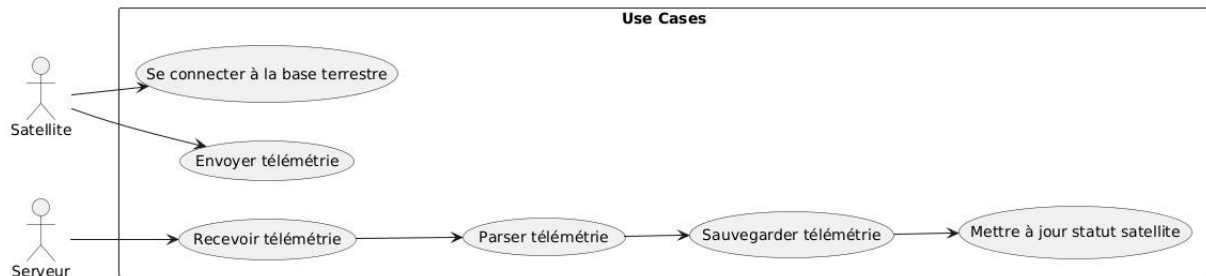
- Code source complet (client + serveur + utils + UI)
- Script SQL pour création des tables
- Exemples de fichiers CSV
- Documentation technique
- Diagrammes UML (classe et séquence)
- Cahier des charges officiel (ce document)

8. Critères de validation

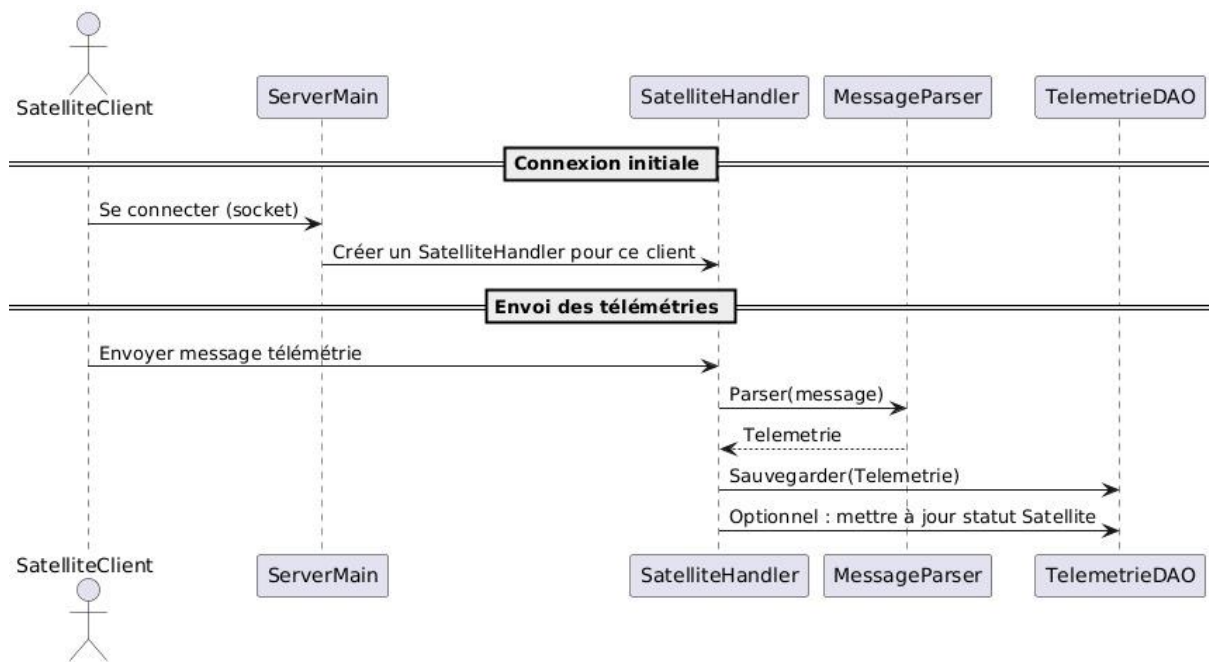
- Le serveur supporte plusieurs satellites simultanément.
- Les télémetries sont correctement reçues, parsées et stockées.
- Données persistées dans SQL et CSV.

Les différents diagrammes UML

1- Diagramme de cas d'utilisation



2- Diagramme de séquence



3- Diagramme de classe

